

Lab 4 : Real-Time Mobile Application Development

Course: Mobile Application Development

Students: Tăng Kim Sơn (22521258), Hồng Bảo Ngọc (22520953)

GitHub: https://github.com/tangkimson/Lab4_Mobile_22521258_22520953

1. Application overview

This lab implements a classic space/chicken shooter game in Kotlin using SurfaceView and a dedicated GameThread running at approximately 60 FPS. The player controls a spaceship at the bottom, shoots yellow bullets at falling enemies, and avoids enemies reaching the bottom boundary.

2. Architecture

- MainActivity : sets GameView as the sole content view
- GameView : central controller: update/draw logic, touch input, HUD
- GameThread : 60 FPS loop: lockCanvas → update → draw → unlockCanvasAndPost
- GameManager : creates opponents, boss, minions, and player ship bitmaps
- Opponent : enemy with lanes, health, and health bar rendering
- FiringObject : yellow bullet rectangles with optional horizontal velocity
- Boss : top alien that moves horizontally and spawns minions

3. Exercise implementations

Screenshots illustrate the implemented features using the project's game assets and HUD layout.

Exercise 1 : Highest score

Requirement: Create an additional area to display the highest score achieved.

Code snippet:

```
// GameView.kt : load and persist high score
private val prefs = context.getSharedPreferences(PREFS_NAME,
Context.MODE_PRIVATE)
```

```
highScore = prefs.getInt(KEY_HIGH_SCORE, 0)
```

```
private fun updateHighScore() {  
    if (score > highScore) {  
        highScore = score  
        prefs.edit().putInt(KEY_HIGH_SCORE, highScore).apply()  
    }  
}
```

```
// draw() : display on HUD
```

```
canvas.drawText("High Score: $highScore", 50f, 130f, hudPaint)
```

Processing logic: On init, the saved high score is read from SharedPreferences. Each time the player scores points or the game ends, `updateHighScore()` compares the current score with the stored value and saves a new record if higher. The `draw()` method renders the value on screen every frame.

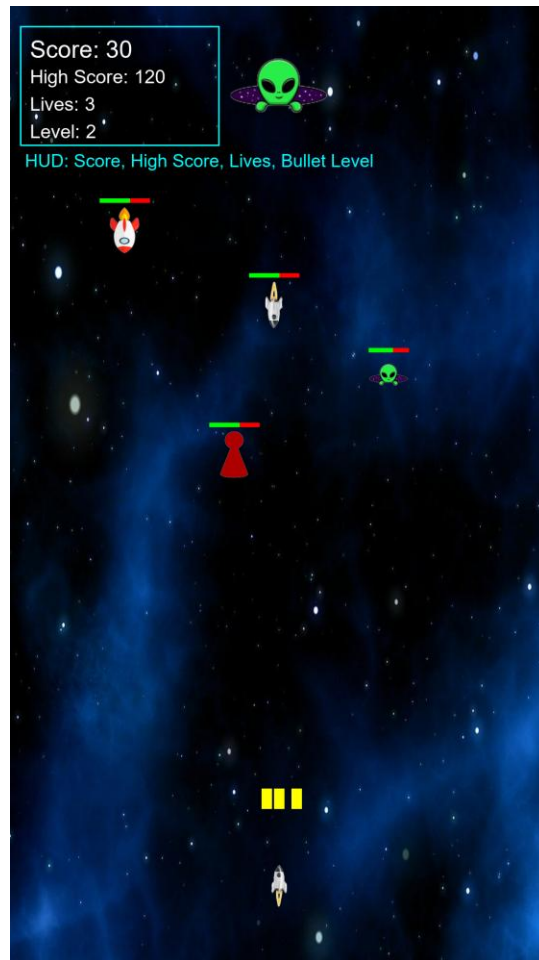


Figure 1: HUD showing current score and highest score

Exercise 2 : Three lives

Requirement: Give the player 3 lives for each game session (the player loses only after enemies reach the boundary 3 times).

Code snippet:

```
// GameView.kt
private var lives: Int = 3

val boundaryY = height - 100f
val reachedBoundary = opponents.filter { it.y + it.height >= boundaryY }
for (opponent in reachedBoundary) {
    opponents.remove(opponent)
    lives--
}
```

```

if (lives <= 0) {
  gameOver = true
  updateHighScore()
}

```

```

// draw()
canvas.drawText("Lives: $lives", 50f, 180f, hudPaint)

```

Processing logic: Each game session starts with 3 lives. In update(), enemies that cross the bottom boundary ($y + \text{height} \geq \text{height} - 100$) are removed and decrement lives. Game over triggers only when lives reaches 0, not on the first boundary breach.

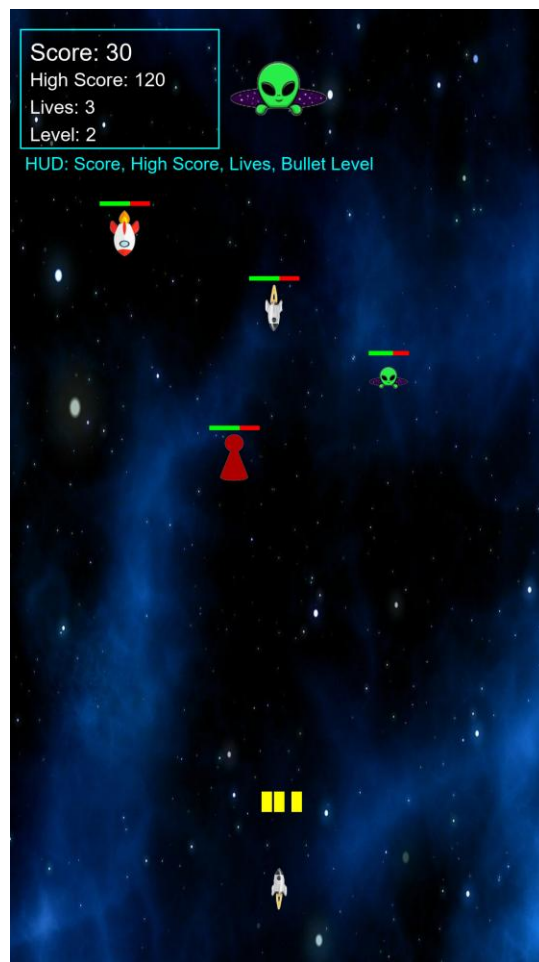


Figure 2: Lives counter displayed on HUD

Exercise 3 : Spaceship graphic

Requirement: Add a spaceship graphic at the shooting position. Each time the player touches the screen, the spaceship should move to the touched location.

Code snippet:

```
// GameManager.kt : create rotated ship bitmap
fun createPlayerShipBitmap(): Bitmap {
    val source = BitmapFactory.decodeResource(context.resources,
R.drawable.rocket_2)
    val scaled = Bitmap.createScaledBitmap(source, 90, 90, false)
    val matrix = Matrix().apply { postRotate(180f) }
    return Bitmap.createBitmap(scaled, 0, 0, scaled.width, scaled.height, matrix,
true)
}

// GameView.kt : onTouchEvent
playerX = (event.x - playerShipWidth / 2f).coerceIn(0f, width - playerShipWidth)
canvas.drawBitmap(playerShipBitmap, null,
    RectF(playerX, playerY, playerX + playerShipWidth, playerY +
playerShipHeight), null)
```

Processing logic: GameManager loads rocket_2.png, scales and rotates it for the player ship. On ACTION_DOWN, playerX is set to the touch X coordinate (clamped to screen). draw() renders the bitmap at (playerX, playerY) above the bottom of the screen.

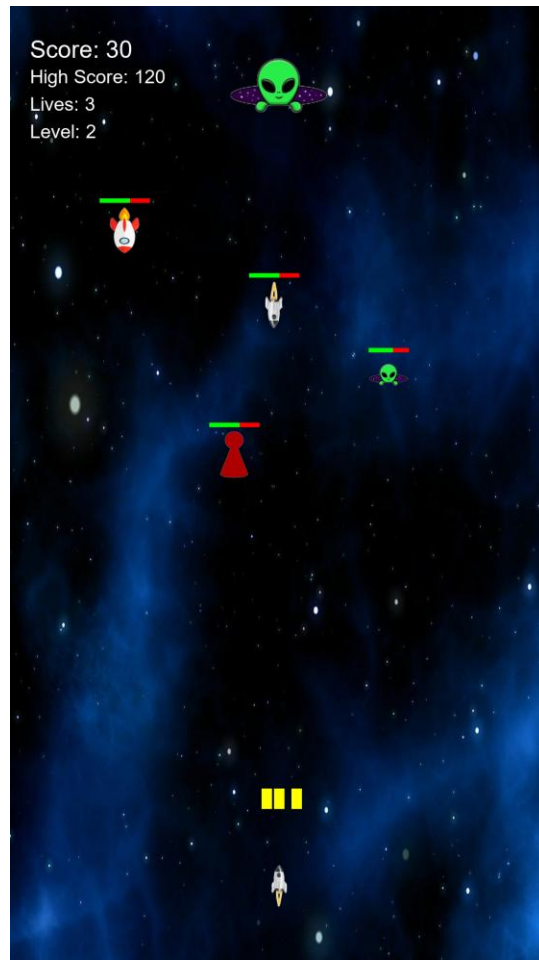


Figure 3: Player spaceship at touch position

Exercise 4 : Spreading bullets

Requirement: Upgrade the bullet system so the bullets can spread/diverge over time. Each upgrade level should fire more bullets.

Code snippet:

```
// GameView.kt
bulletLevel = (1 + score / 50).coerceAtMost(5)

private fun fireBullets(centerX: Float, centerY: Float) {
    when (bulletLevel) {
        1 -> addBullet(centerX, centerY, 0f)
        2 -> {
            addBullet(centerX, centerY, 0f)
            addBullet(centerX - 25f, centerY, -3f)
```

```

        addBullet(centerX + 25f, centerY, 3f)
    }
    // levels 3–5 add more bullets with wider vx spread
}
}

```

```

// FiringObject.kt : horizontal velocity
fun update() { x += vx; y -= speed; ... }

```

Processing logic: bulletLevel increases every 50 points (max 5). fireBullets() uses a when expression to fire 1–5 bullets per tap, each with optional horizontal velocity (vx). FiringObject.update() applies vx so bullets spread outward as level rises.

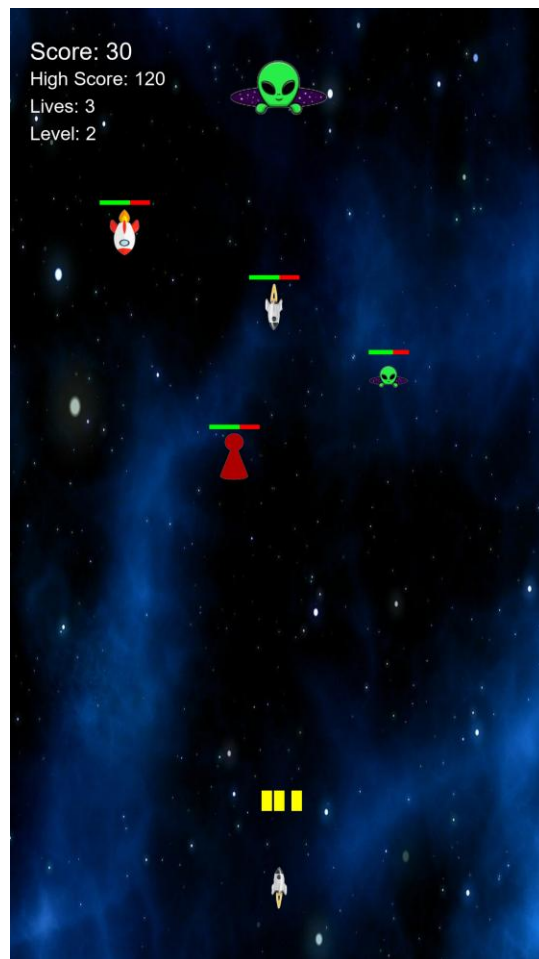


Figure 4: Multiple spread bullets fired from the ship

Exercise 5 : Enemy health bars

Requirement: Add a health bar for each enemy. Every enemy should have a random amount of health, and the health bar should gradually decrease whenever the enemy is hit.

Code snippet:

```
// Opponent.kt
init {
    maxHealth = (2..5).random()
    currentHealth = maxHealth
}

fun takeDamage(damage: Int): Boolean {
    currentHealth -= damage
    return currentHealth <= 0
}

private fun drawHealthBar(canvas: Canvas) {
    val healthRatio = currentHealth.toFloat() / maxHealth.toFloat()
    canvas.drawRect(left, top, left + barWidth, top + barHeight, healthBgPaint)
    canvas.drawRect(left, top, left + barWidth * healthRatio, top + barHeight,
healthFgPaint)
}
```

Processing logic: Each Opponent spawns with random maxHealth (2–5). On bullet collision, takeDamage(1) reduces currentHealth. The enemy is removed only when health reaches 0. drawHealthBar() draws a red background and green foreground bar whose width reflects remaining health.

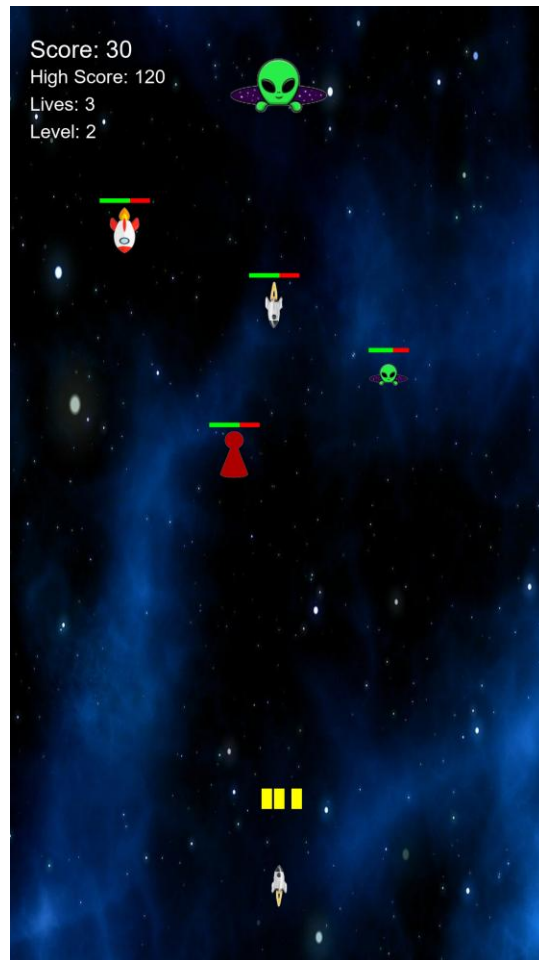


Figure 5: Enemies with health bars above them

Exercise 6 : Lane movement

Requirement: Make enemies capable of moving across different lanes instead of only moving straight downward.

Code snippet:

```
// Opponent.kt
companion object { const val LANE_COUNT = 5 }

private val laneWidth: Float = screenWidth.toFloat() / laneCount
private val laneCenterX: Float = laneIndex * laneWidth + laneWidth / 2f - width / 2f

fun update() {
    y += speed
}
```

```

val diff = laneCenterX - x
if (abs(diff) > 1f) {
    x += diff.coerceIn(-4f, 4f)
}
rect.set(x, y, x + width, y + height)
}

```

Processing logic: The screen is divided into 5 lanes. Each enemy has a laneIndex and a laneCenterX target. While falling ($y += \text{speed}$), the enemy drifts horizontally toward its lane center at up to 4 px/frame, creating diagonal lane movement.

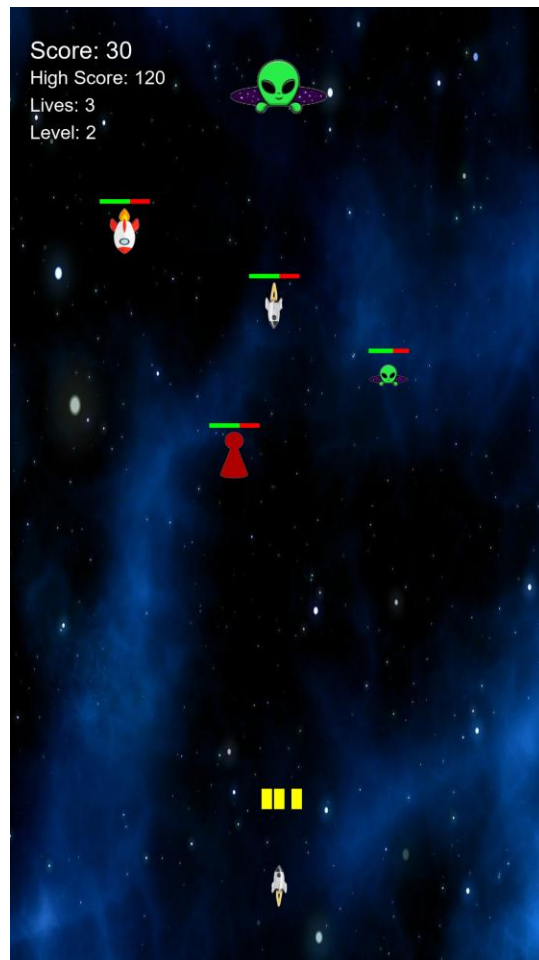


Figure 6: Enemies drifting across lanes while falling

Exercise 7 : Boss alien

Requirement: Create a larger alien enemy (Boss) that moves left and right while remaining fixed near the top of the screen. This Boss should spawn smaller aliens from the center of the Boss instead of spawning enemies randomly.

Code snippet:

```
// Boss.kt
fun update(screenWidth: Int) {
    x += dx
    if (x <= 0f || x + width >= screenWidth) { dx = -dx }
}

fun shouldSpawn(): Boolean {
    spawnTimer++
    if (spawnTimer >= spawnInterval) { spawnTimer = 0; return true }
    return false
}

// GameView.kt : boss spawns minions (no random spawn)
boss?.let { currentBoss ->
    currentBoss.update(width)
    if (currentBoss.shouldSpawn()) {
        val minion = gameManager.createMinion(
            currentBoss.getCenterX(), currentBoss.getCenterY() + ..., ...)
        opponents.add(minion)
    }
}
```

Processing logic: Boss is created at y=60 and moves horizontally, bouncing off screen edges. Every 90 frames, shouldSpawn() returns true and GameView calls GameManager.createMinion() at the boss center. Random enemy spawning was removed from update(); all enemies now come from the boss.

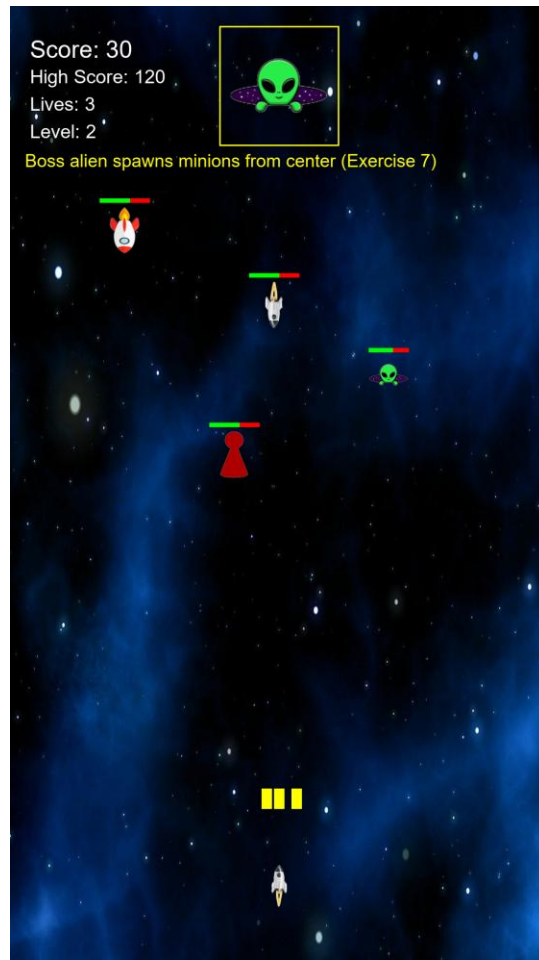


Figure 7: Boss alien at top spawning minions

4. Game over screen

When lives reach zero, the game displays 'Game Over' and 'Tap to restart'.

Tapping the screen calls `resetGame()` to clear objects and restore initial values.



Figure 8: Game Over screen

5. How to run the project

1. Clone: git clone
https://github.com/tangkimson/Lab4_Mobile_22521258_22520953.git
2. Open the project folder in Android Studio on Windows.
3. Wait for Gradle sync (Android Studio creates local.properties automatically).
4. Connect an emulator (API 24+) or physical device.
5. Run the app (Shift+F10) or build with: `.\gradlew.bat assembleDebug`
6. Touch the screen to move the ship and fire. Tap after Game Over to restart.